

Testing APIs with Postman Assignment.

Name: Ineza Sugira Aimee Marylene

Date: 5th September 2029

1. API Choice

For this assignment, I used the GitHub REST API and GitHub Gists to demonstrate API testing and CRUD operations. This was because it provides authenticated API access and supports the operations required to test. GitHub Gists were chosen because they support all four CRUD operations required ; POST to create, GET to read, PATCH to update, and DELETE to remove a Gist.

base URL: <https://api.github.com>

Resource: GitHub Gists

API Endpoints

Operation	Method	Endpoint
Authentication	GET	{{base_url}}/user
Create Gist	POST	{{base_url}}/gists
Retrieve Gist	GET	{{base_url}}/gists/{{gist_id}}
Update Gist	PATCH	{{base_url}}/gists/{{gist_id}}
Delete Gist	DELETE	{{base_url}}/gists/{{gist_id}}

2. Postman Environment

The Postman environment contains variables used throughout the API requests.

- **Base_url:** <https://api.github.com>
- **auth_token** – Authentication token
- **gist_id** – ID of the created Gist

API Assignment Fork 0 Share

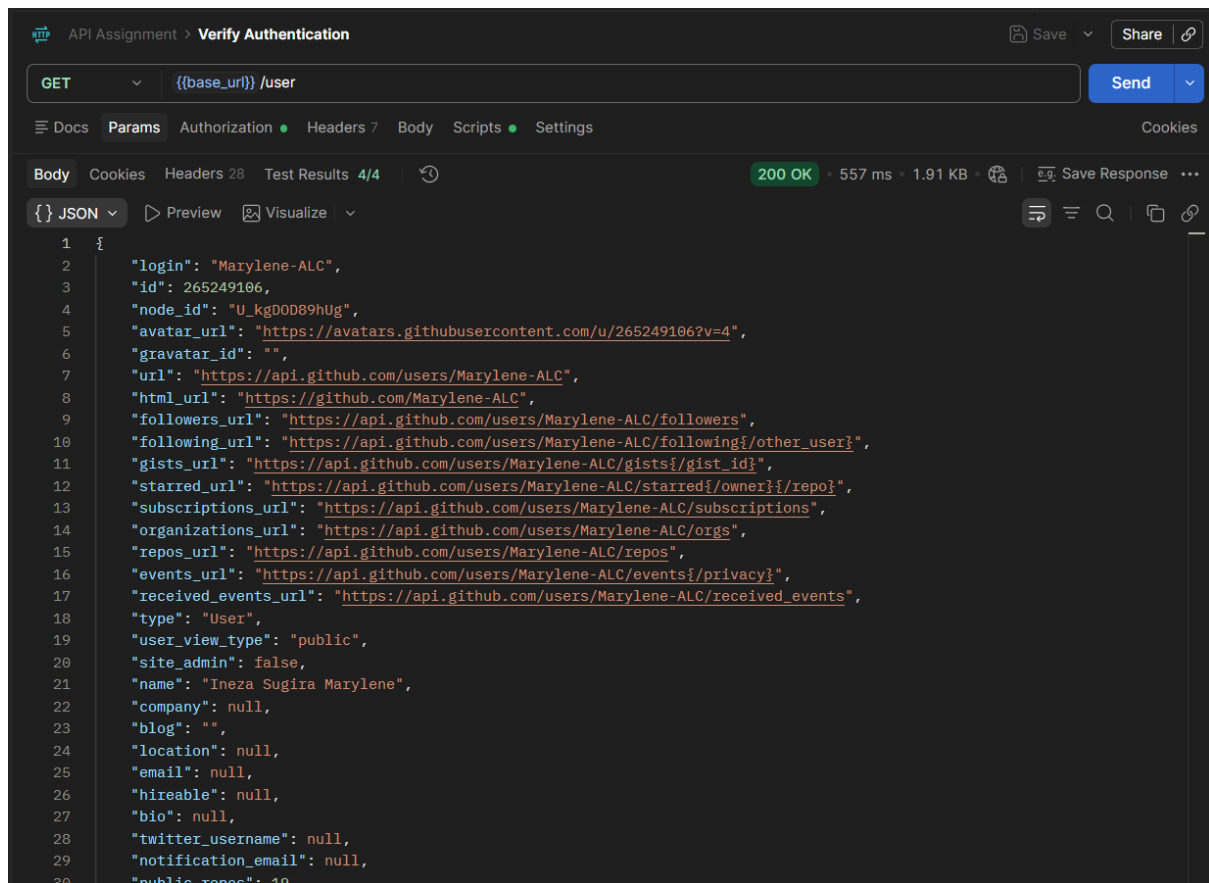
Variable	Value	
base_url	https://api.github.com	
auth_token	github_personal_access_token_0svi	Share
gist_id	56b16f373fcba70b8b7603b588cf17dd	
Add variable		

3. Authentication

For Authentication, I used my GitHub Personal Access Token (PAT). The token was stored in the `auth_token` Postman environment variable and I used it as a Bearer Token for all the requests. A pre-request script was not needed to fetch the token, since GitHub Personal Access Tokens are generated once in GitHub settings, not through a login request. The token was simply stored in the `auth_token` environment variable and reused for all requests as a Bearer Token.

Authentication Request

GET: `{{base_url}}/user`

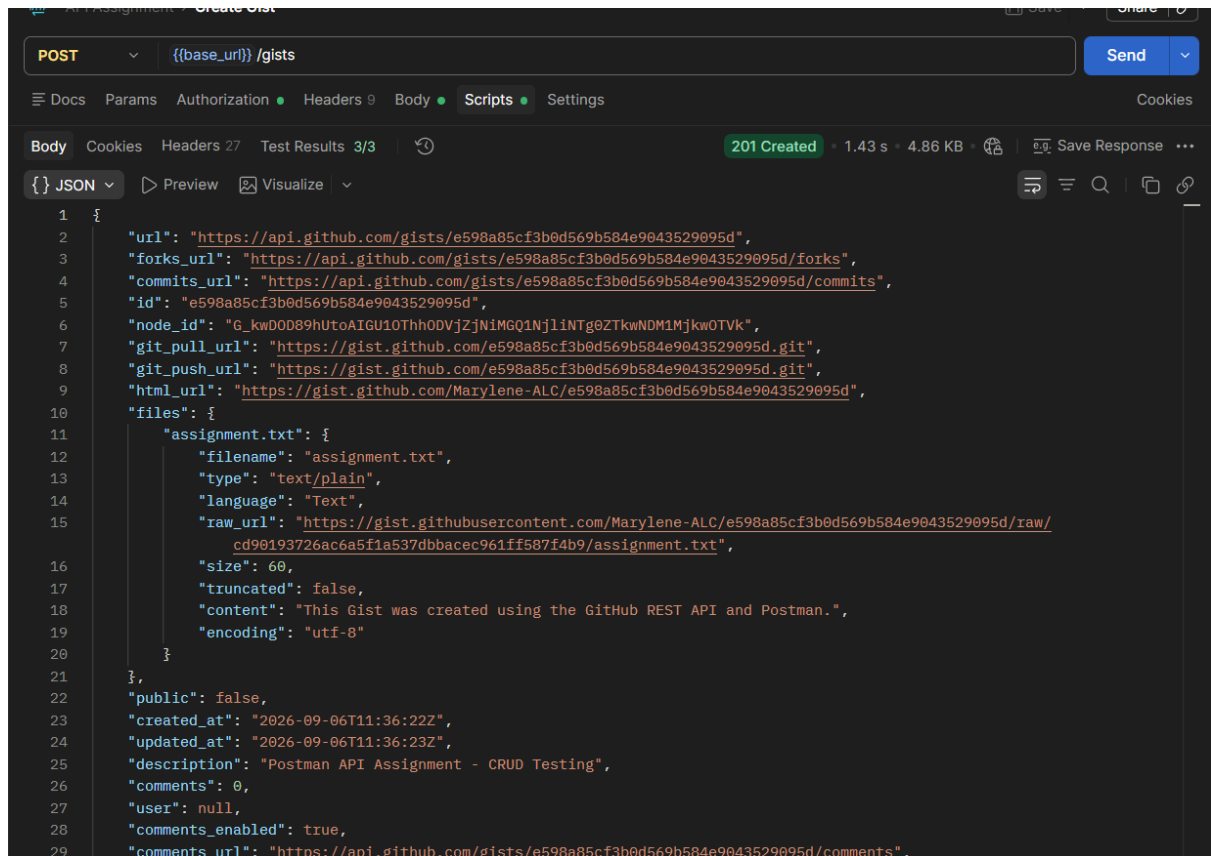


4. CRUD Operations

4.1 Create Gist - POST

A POST request was used to create a new Gist.

POST: {{base_url}}/gists



```
POST {{base_url}} /gists

201 Created • 1.43 s • 4.86 KB • Save Response ...

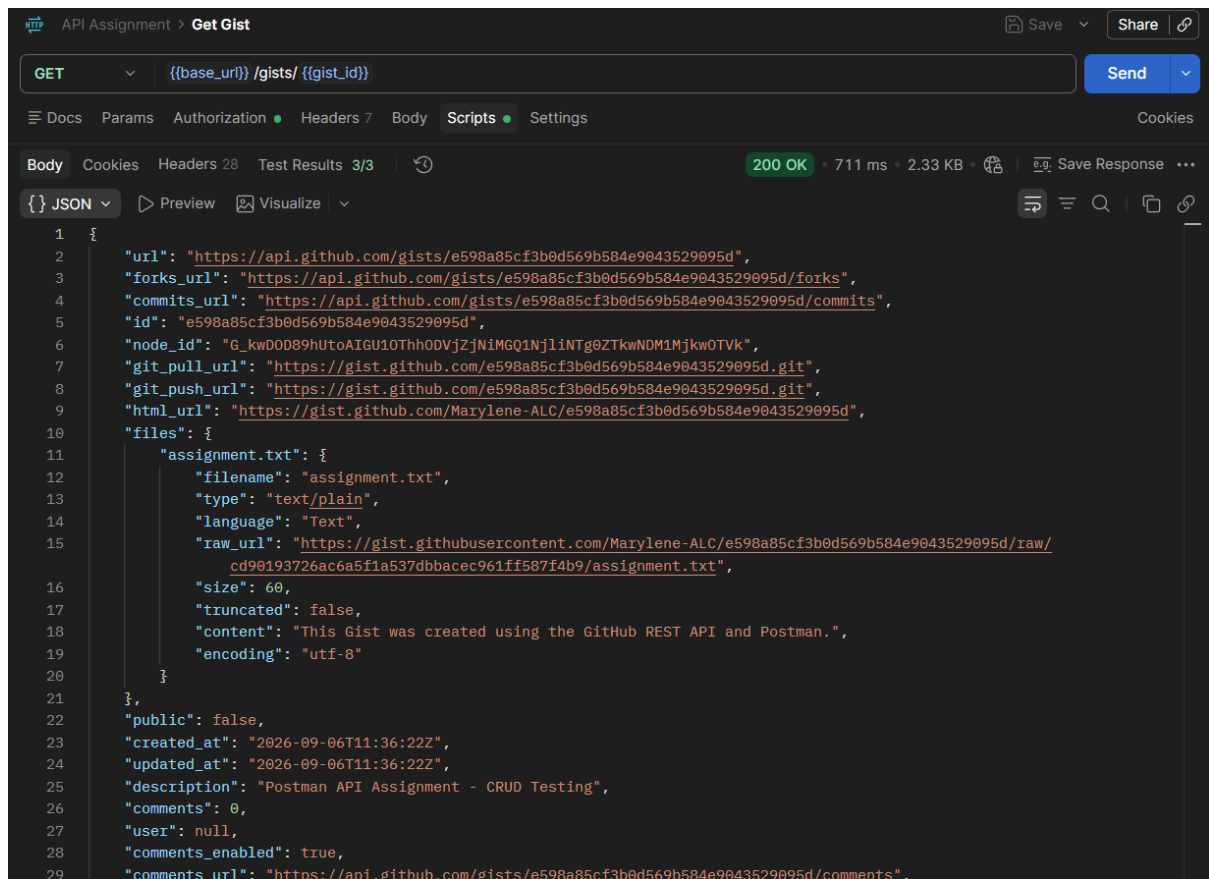
{} JSON Preview Visualize

1 {
2   "url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d",
3   "forks_url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d/forks",
4   "commits_url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d/commits",
5   "id": "e598a85cf3b0d569b584e9043529095d",
6   "node_id": "G_kwD0D89hUtoAIGU10Thh0DVjZjNmMGQ1NjliNTg0ZTkWNDM1MjkwOTVh",
7   "git_pull_url": "https://gist.github.com/e598a85cf3b0d569b584e9043529095d.git",
8   "git_push_url": "https://gist.github.com/e598a85cf3b0d569b584e9043529095d.git",
9   "html_url": "https://gist.github.com/Marylene-ALC/e598a85cf3b0d569b584e9043529095d",
10  "files": {
11    "assignment.txt": {
12      "filename": "assignment.txt",
13      "type": "text/plain",
14      "language": "Text",
15      "raw_url": "https://gist.githubusercontent.com/Marylene-ALC/e598a85cf3b0d569b584e9043529095d/raw/cd90193726ac6a5f1a537dbbacec961ff587f4b9/assignment.txt",
16      "size": 60,
17      "truncated": false,
18      "content": "This Gist was created using the GitHub REST API and Postman.",
19      "encoding": "utf-8"
20    }
21  },
22  "public": false,
23  "created_at": "2026-09-06T11:36:22Z",
24  "updated_at": "2026-09-06T11:36:23Z",
25  "description": "Postman API Assignment - CRUD Testing",
26  "comments": 0,
27  "user": null,
28  "comments_enabled": true,
29  "comments_url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d/comments",
```

4.2 Retrieve Gist - GET

A GET request was used to retrieve the created Gist.

GET: `{{base_url}}/gists/{{gist_id}}`



API Assignment > Get Gist

GET {{base_url}}/gists/{{gist_id}}

200 OK • 711 ms • 2.33 KB

```
1 {
2   "url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d",
3   "forks_url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d/forks",
4   "commits_url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d/commits",
5   "id": "e598a85cf3b0d569b584e9043529095d",
6   "node_id": "G_kwDOD89hUtoAIGU10ThhODVjZjNiMGQ1NjliNTg0ZTkwNDM1MjkwOTVh",
7   "git_pull_url": "https://gist.github.com/e598a85cf3b0d569b584e9043529095d.git",
8   "git_push_url": "https://gist.github.com/e598a85cf3b0d569b584e9043529095d.git",
9   "html_url": "https://gist.github.com/Marylene-ALC/e598a85cf3b0d569b584e9043529095d",
10  "files": {
11    "assignment.txt": {
12      "filename": "assignment.txt",
13      "type": "text/plain",
14      "language": "Text",
15      "raw_url": "https://gist.githubusercontent.com/Marylene-ALC/e598a85cf3b0d569b584e9043529095d/raw/cd90193726ac6a5f1a537dbbacec961ff587f4b9/assignment.txt",
16      "size": 60,
17      "truncated": false,
18      "content": "This Gist was created using the GitHub REST API and Postman.",
19      "encoding": "utf-8"
20    }
21  },
22  "public": false,
23  "created_at": "2026-09-06T11:36:22Z",
24  "updated_at": "2026-09-06T11:36:22Z",
25  "description": "Postman API Assignment - CRUD Testing",
26  "comments": 0,
27  "user": null,
28  "comments_enabled": true,
29  "comments_url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d/comments",
```

4.3 Update Gist - PATCH

A PATCH request was used to update the Gist.

PATCH: `{{base_url}}/gists/{{gist_id}}`

The screenshot shows a Postman interface for a PATCH request to the endpoint `{{base_url}}/gists/{{gist_id}}`. The request is successful, returning a **200 OK** status with a response time of 1.02 s and a body size of 2.36 KB. The response body is displayed in JSON format, showing the updated Gist details. The JSON includes fields for `url`, `forks_url`, `commits_url`, `id`, `node_id`, `git_pull_url`, `git_push_url`, `html_url`, `files` (containing an `assignment.txt` file), `public`, `created_at`, `updated_at`, `description`, `comments`, `user`, `comments_enabled`, and `comments_url`.

```
1 {
2   "url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d",
3   "forks_url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d/forks",
4   "commits_url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d/commits",
5   "id": "e598a85cf3b0d569b584e9043529095d",
6   "node_id": "G_kwD0D89hUtoAIGU10Thh0DVjZjNiMGQ1NjliNTg0ZTkwNDM1MjkwOTVh",
7   "git_pull_url": "https://gist.github.com/e598a85cf3b0d569b584e9043529095d.git",
8   "git_push_url": "https://gist.github.com/e598a85cf3b0d569b584e9043529095d.git",
9   "html_url": "https://gist.github.com/Marylene-ALC/e598a85cf3b0d569b584e9043529095d",
10  "files": {
11    "assignment.txt": {
12      "filename": "assignment.txt",
13      "type": "text/plain",
14      "language": "Text",
15      "raw_url": "https://gist.githubusercontent.com/Marylene-ALC/e598a85cf3b0d569b584e9043529095d/raw/d7ed5b82cff5ec9c03d702bb504e9b0451a90812/assignment.txt",
16      "size": 55,
17      "truncated": false,
18      "content": "This Gist was updated using a PATCH request in Postman.",
19      "encoding": "utf-8"
20    }
21  },
22  "public": false,
23  "created_at": "2026-09-06T11:36:22Z",
24  "updated_at": "2026-09-06T12:22:50Z",
25  "description": "Postman API Assignment - Updated Gist",
26  "comments": 0,
27  "user": null,
28  "comments_enabled": true,
29  "comments_url": "https://api.github.com/gists/e598a85cf3b0d569b584e9043529095d/comments",
```

4.4 Delete Gist - DELETE

A DELETE request was used to remove the Gist.

DELETE: `{{base_url}}/gists/{{gist_id}}`

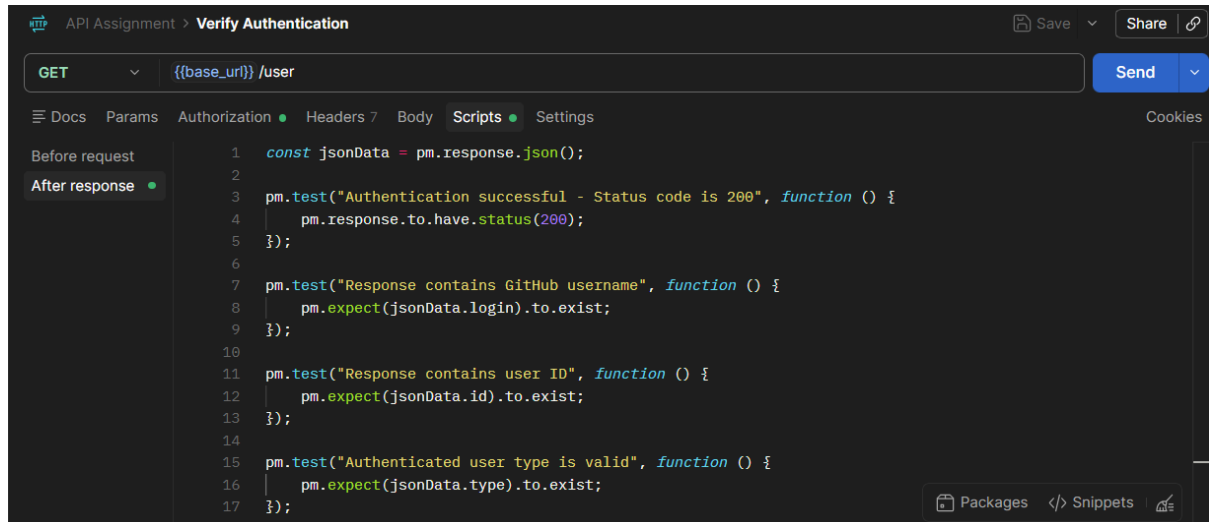
The screenshot shows a Postman interface for a DELETE request to the endpoint `{{base_url}}/gists/{{gist_id}}`. The request is successful, returning a **204 No Content** status with a response time of 455 ms and a body size of 1.14 KB. The response body is displayed in Raw format, showing a single line of text.

```
1
```

5. Automation and Testing

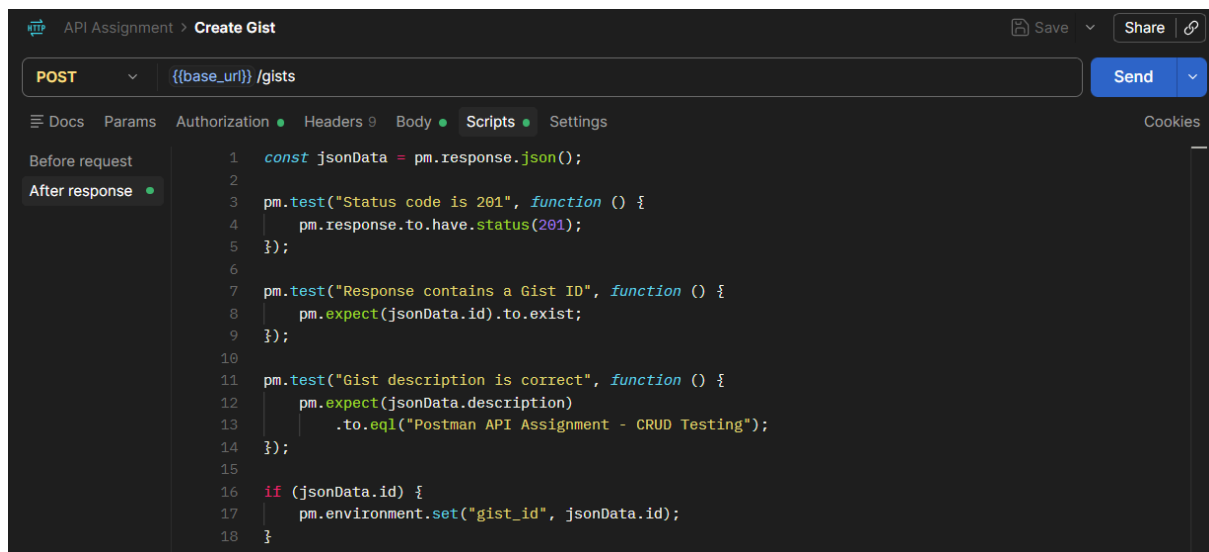
Postman test scripts were used to automatically check response status codes and response data.

The `gist_id` was also automatically saved as an environment variable after creating a Gist. A simple pre-request script was added to check that the `auth_token` exists before sending each request.



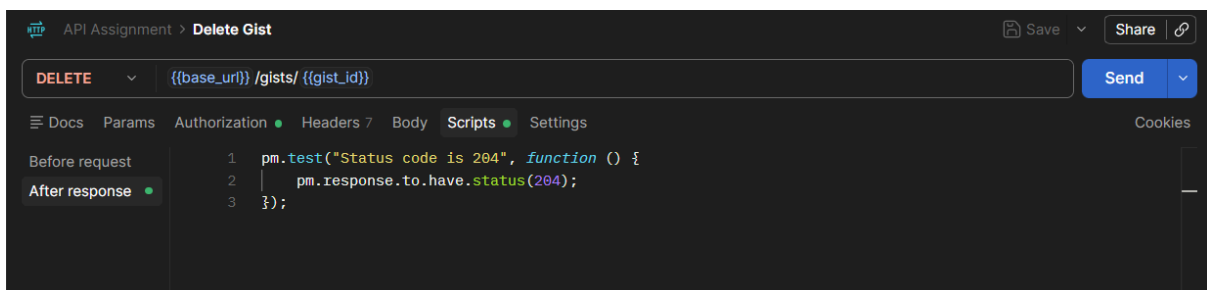
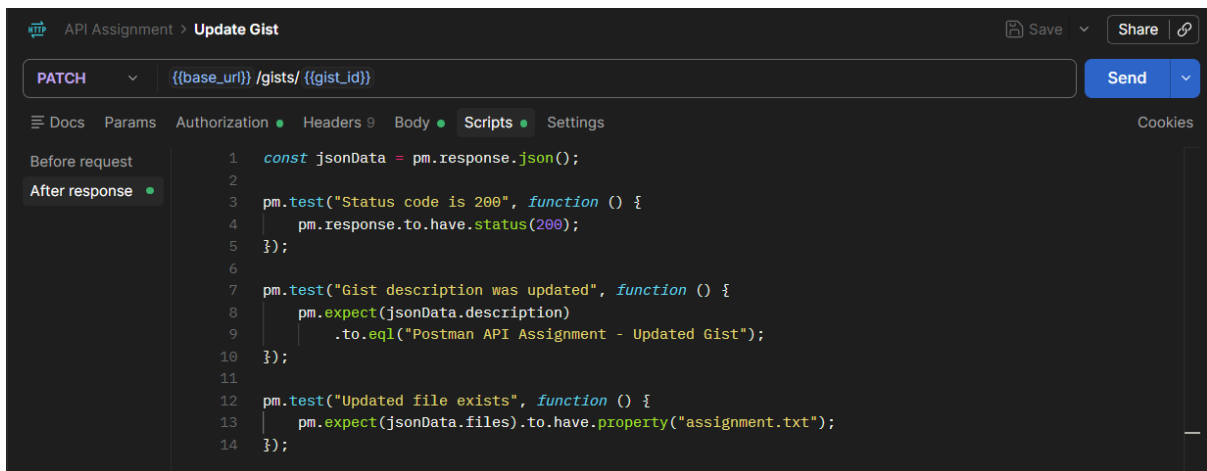
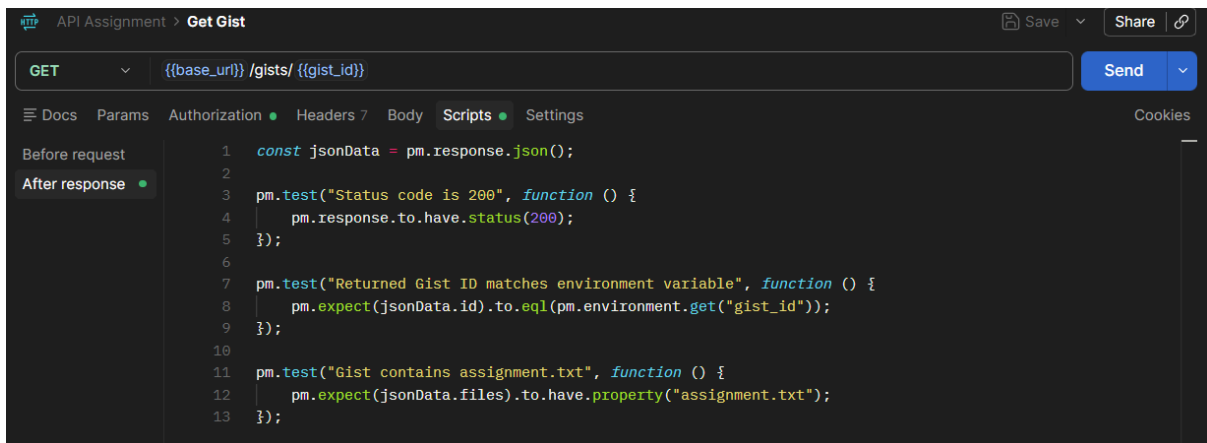
The screenshot shows the Postman interface for an API endpoint named "Verify Authentication". The method is GET and the URL is `{{base_url}} /user`. The "Scripts" tab is selected, showing a test script for the "After response" event. The script checks the status code and the presence of user information in the response.

```
1 const jsonData = pm.response.json();
2
3 pm.test("Authentication successful - Status code is 200", function () {
4   pm.response.to.have.status(200);
5 });
6
7 pm.test("Response contains GitHub username", function () {
8   pm.expect(jsonData.login).to.exist;
9 });
10
11 pm.test("Response contains user ID", function () {
12   pm.expect(jsonData.id).to.exist;
13 });
14
15 pm.test("Authenticated user type is valid", function () {
16   pm.expect(jsonData.type).to.exist;
17 });
```



The screenshot shows the Postman interface for an API endpoint named "Create Gist". The method is POST and the URL is `{{base_url}} /gists`. The "Scripts" tab is selected, showing a test script for the "After response" event. The script checks the status code, the presence of a Gist ID, and the correctness of the Gist description. It also sets the `gist_id` as an environment variable.

```
1 const jsonData = pm.response.json();
2
3 pm.test("Status code is 201", function () {
4   pm.response.to.have.status(201);
5 });
6
7 pm.test("Response contains a Gist ID", function () {
8   pm.expect(jsonData.id).to.exist;
9 });
10
11 pm.test("Gist description is correct", function () {
12   pm.expect(jsonData.description)
13     .to.eql("Postman API Assignment - CRUD Testing");
14 });
15
16 if (jsonData.id) {
17   pm.environment.set("gist_id", jsonData.id);
18 }
```



6. Collection Runner

The Collection Runner was used to automatically run the requests in sequence:

Authentication → Create → Retrieve → Update → Delete → Verify Deletion

API Assignment - Run results

Ran today at 04:45:06 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	API Assignment	1	4s 22ms	14	0	649 ms

All 14 Passed 14 Failed 0 Skipped 0 Errors 0 Console log

Iteration 1

GET Verify Authentication
https://api.github.com/user
200 • 579 ms • 1.971 KB 4

- PASS Authentication successful - Status code is 200
- PASS Response contains GitHub username
- PASS Response contains user ID
- PASS Authenticated user type is valid

POST Create Gist
https://api.github.com/gists
201 • 1020 ms • 5.019 KB 3

- PASS Status code is 201
- PASS Response contains a Gist ID
- PASS Gist description is correct

GET Get Gist
https://api.github.com/gists/54c364aa8e9ca40619e0f28d529389ed
200 • 460 ms • 2.402 KB 3

- PASS Status code is 200
- PASS Returned Gist ID matches environment variable
- PASS Gist contains assignment.txt

PATCH Update Gist
https://api.github.com/gists/54c364aa8e9ca40619e0f28d529389ed
200 • 816 ms • 2.43 KB 3

- PASS Status code is 200
- PASS Gist description was updated
- PASS Updated file exists

DELETE Delete Gist
https://api.github.com/gists/54c364aa8e9ca40619e0f28d529389ed
204 • 368 ms • 1.21 KB 1

- PASS Status code is 204

7. Headers and Body.

Operation	Method	Endpoint	Headers	Body
Authentication	GET	{{base_url}}/user	Authorization: Bearer {{auth_token}}	

Create Gist	POST	{{base_url}}/gists	Authorization: Bearer {{auth_token}}, Content-Type: application/json	{ "description": "...", "public": false, "files": {...} }
Retrieve Gist	GET	{{base_url}}/gists/{{gist_id}}	Authorization: Bearer {{auth_token}}	
Update Gist	PATCH	{{base_url}}/gists/{{gist_id}}	Authorization: Bearer {{auth_token}}, Content-Type: application/json	{ "description": "...", "files": {...} }
Delete Gist	DELETE	{{base_url}}/gists/{{gist_id}}	Authorization: Bearer {{auth_token}}	